
Estructuras

Introducción

- *Las estructuras son colecciones de variables relacionadas.*
- *Con los arreglos hemos visto un tipo de datos compuesto, en el que todos los elementos son de igual tipo y pueden ser ubicados mediante un subíndice.*
- *Una estructura, que también es un tipo de datos compuesto tiene elementos que por lo general son de distinto tipo.*

Introducción

- *Es decir, estamos encerrando bajo un mismo "envoltorio" variables que describen las características de una unidad o un elemento.*
- *Por ejemplo las personas tienen un nombre (string), una edad (integer) una estatura (float).*
- *Para relacionar estos elementos con una persona utilizamos una estructura.*

Introducción

- *Las estructuras son el elemento ideal para definir registros a almacenarse en archivos.*
- *Con apuntadores y estructuras se pueden generar construcciones muy interesantes como son las listas enlazadas, pilas, colas y árboles.*
- *Estas últimas son conocidas habitualmente como: estructuras de datos.*

Declaración

Forma general:

```
struct etiqueta {
  tipo nombre_var;
  tipo nombre_var;
  tipo nombre_var;
  ...
} variables_struct;
```

- ***struct*** *es la palabra reservada que se utiliza para la declaración.*

Declaración

- *La* `etiqueta` *sirve para darle un nombre al molde para generar variables de esta estructura, de la forma:*

```
struct etiqueta var1, var2[10];
```

- `variables_struct` *son variables del tipo de la estructura que están definidas junto con la declaración.*
- `etiqueta y variables_struct` *pueden omitirse, no simultáneamente.*

Declaración

- *La declaración de una estructura no reserva ningún espacio en memoria.*
- *Lo que realmente genera es un nuevo tipo de datos.*
- *Este nuevo tipo es el que utilizamos para definir variables que tendrán adelante la palabra reservada:*

`struct`

Ejemplo de declaración

```
struct lindir {
  char nom[9];
  char ext[4];
  long int longitud;
  char fecha[9];
  int hora, minutos;
};
struct lindir ld1, ld2, ld[50];
```

- *En este caso se declara la estructura sin definir variables, lo que se hace en la sentencia siguiente.*

Ejemplo de declaración

```
struct {
  char nom[9];
  char ext[4];
  long int longitud;
  char fecha[9];
  int hora, minutos;
} linea;
```

- *Aquí no le hemos dado nombre a la estructura, pero sí hemos definido una variable. No podemos definir más variables de este tipo.*

Utilización de typedef

- ***typedef*** *es una palabra reservada que me permite generar un sinónimo de un tipo existente o al declararlo.*
- *Si hemos declarado una estructura, p. ej. sbase, de la siguiente manera:*

```
struct sbase { ........ };
```

- *Para definir variables de esa estructura debemos indicar:*

```
struct sbase var1, var2, var3;
```

Utilización de typedef

- *Con typedef generamos un sinónimo de la siguiente manera:*

```
typedef struct sbase base;
```

- *Ahora base, que es un sinónimo de struct sbase, se comporta como un nuevo tipo:*

```
base var1, var2, var3;
```

- *Esto no cambió el concepto, sólo generó un sinónimo.*

Utilización de typedef

- *También podemos utilizar el typedef en el mismo momento que estamos declarando la estructura, p. ej.:*

```
typedef struct sbase {
   ...
   ...
} base;
```

- *Ahora utilizamos* `base` *para definir variables de esa estructura.*

Utilización de typedef

- *Para abreviar más aún se puede omitir el nombre de modelo de la estructura, esto es:*

```
typedef struct {
   ...
   ...
} base;
```

- `base` *está en condiciones de ser utilizado para definir variables.*

Referencia a los elementos

- *Para referirnos a un elemento en particular de la estructura (campo), utilizamos la notación:*

```
variab_estructura.campo
```

- *Es decir, que debemos escribir el nombre de la variable de estructura seguido de un punto y a continuación, sin dejar espacios en blanco, el nombre del campo a que hacemos referencia.*

Declaración e inicialización

- *Como sucede con cualquier tipo de variable, podemos definir e inicializar una estructura en una sola sentencia.*

```
struct lindir linea2 = { "CAMBIO",
   "EXE", 23895, "27/09/99", 16, 09 };
```

- *Esto nos recuerda la definición e inicialización simultánea de arreglos, simplemente que en este caso hay que asignar elementos de distinto tipo.*

Asignación de campos

- *En el medio de un programa debemos utilizar las sentencias adecuadas para el cambio de valor de las variables, p. ej:*

```
strcpy(lindir.ext, "COM");
```

- *En síntesis, un campo de estructura se comporta exactamente igual como lo haría una variable simple del mismo tipo y son totalmente compatibles.*

Anidamiento de estructuras

- *Dentro de una estructura podemos declarar otras estructuras.*
- *Esto se llama: anidamiento de estructuras.*
- *Cuando nos referimos a un elemento de una estructura anidada, separamos cada uno de los nombres de variables de estructura con puntos, hasta terminar con el nombre del campo buscado.*

```
struct sfecha {
  int dia, mes, anio;
}

struct shora {
  int horas, minutos;
}

struct lindir {
  char nom[9];
  char ext[4];
  long int longitud;
  struct sfecha fecha;
  struct shora hora;
} prinlin;

prinlin.fecha.dia = 23;
```

Arreglos de estructuras

- *Una estructura aislada no es de mucha utilidad, lo común es tener arreglos de estructuras o punteros a estructuras.*
- *La definición de un arreglo de estructuras se hace así:*

```
struct lindir director[100];
```

- *La asignación de un elemento de esta estructura se hace así.*

```
director[7].fecha.mes = 4;
```

Punteros a estructuras

- *Se pueden establecer punteros a estructuras al igual que los punteros a cualquier otra clase de variable.*

```
struct lindir *scrlin;
```

- *Los punteros a estructuras se utilizan por dos motivos principales: a) pasaje de estructuras por referencia, b) generación de listas enlazadas y árboles utilizando la memoria dinámica.*

Punteros a estructuras

- *Para referirnos a un elemento de esta estructura utilizamos el operador flecha*

```
linea->longitud = 512000;
```

- *El operador flecha equivale al punto en las variables comunes de estructura.*
- *Es la notación más utilizada, la que es un acortamiento de:*

```
(*linea).longitud = 512000;
```

Pasaje de estructuras a funciones

- *Las estructuras pasan por **valor** a las funciones.*
- *A menos que pasemos un puntero a estructura, en cuyo caso pasa por **referencia**.*
- *Una forma, quizás no ortodoxa, de pasar un arreglo por valor a una función, es envolverlo con una estructura.*
- *Esto último debe usarse con cuidado.*